单词记忆 WordMemory 开发文档

项目	内容
课程	《Android 移动终端应用开发技术》（2 学分）+ 《移动互联网应用开发实战》（1 学分）
选题	主 App：B4 单词记忆；伴生模块：P2 数据看板
学号 / 姓名	202405100117 / 王洲
代码仓库	https://github.com/wwdcqxt/wangzhou-design
完成时间	2026-11

1. 项目概述

本项目包含两个可独立运行、相互关联的成果：

1. **主 App「单词记忆」**：以艾宾浩斯遗忘曲线为核心的自定义算法 App，覆盖复习队列、词库导入、测试模式、错题回顾与每日提醒；
2. **伴生模块「单词看板」**：独立 APK，通过 ContentProvider 读取主 App 的统计数据，用 Canvas 自绘图表展示学习情况，并支持导出 CSV。

两者共用同一个仓库、同一个 `core` 共享层，但安装后是两个独立应用，均可单独演示。

选题满足 B4 的全部强制技术点（自定义算法 + 通知），并额外实现了网络请求（免费词典 API）与后台任务（WorkManager），

因此在「最低复杂度标准」的硬能力一项上有两个落点。

2. 需求分析

2.1 用户场景

- 备考的学生需要在碎片时间背单词，希望系统**自动安排每天该复习哪些词**，而不是自己判断；
- 词库来源多样（老师发的 CSV、自己整理的 TXT、网上下载的 JSON），需要**能导入**；
- 背过的词会反复忘，需要**错题/易错词单独沉淀**；
- 需要看到自己的坚持程度（每日复习量、连续打卡、正确率），并能把数据**导出**交给老师或做周报。

2.2 功能需求

编号	功能	优先级	对应实现
F1	今日复习队列（按到期时	必做	`nav_home` 首页 +

	间排序）		`ReviewDao.observeDueWords`
F2	四档评分并计算下次复习时间	必做	`EbbinghausScheduler.next`
F3	词库导入（CSV/TXT/JSON）	必做	`WordBookParser` + `ImportFragment`
F4	词库/单词浏览与搜索、单词详情	必做	`nav_wordbooks` → `nav_wordlist` → `nav_worddetail`
F5	测试模式（四选一）	必做	`QuizFragment`
F6	错题回顾	自选	`WrongBookFragment` + `wrong_records` 表
F7	每日复习提醒	必做（B4 强制点）	`ReminderWorker` + `NotificationHelper`
F8	在线补充释义	加分项	`DictionaryRepository`（dictionaryapi.dev）
F9	统计看板与 CSV 导出	伴生模块	`dashboard` 模块

2.3 非功能需求

- **三态完整**：每个异步页面都有加载 / 空 / 错误三态，错误态可重试；
- **权限降级**：通知权限被拒绝时，提醒功能退化为应用内横幅，不崩溃、不阻塞主流程；
- **主线程零阻塞**：所有数据库与网络操作在 IO 线程，UI 只收集 Flow；
- **数据一致性**：导入词库、提交复习都是事务操作，失败整体回滚；
- **离线可用**：除「在线释义」外，全部功能离线可用。

3. 功能结构与页面导航

```
底部导航
├── 今日（HomeFragment）
│     ├── 复习会话（ReviewFragment）
│     └── 测试模式（QuizFragment）
├── 词库（WordBookFragment）
│     ├── 单词列表（WordListFragment）──→ 单词详情（WordDetailFragment）
│     └── 导入词库（ImportFragment）
├── 错题（WrongBookFragment）
│     ├── 单词详情（WordDetailFragment）
│     └── 错题再练（QuizFragment，wrongOnly=true）
└── 我的（SettingsFragment）
```

共 9 个界面，其中「词库 → 单词列表 → 单词详情」构成完整的列表→详情导航链路。

底部导航只在 4 个一级页面显示，进入二级页面自动隐藏。

4. 系统设计

4.1 模块划分

```
:app                界面层（Fragment + ViewModel + Adapter）
  └── :core         数据层 + 算法层 + 契约层（无 UI 依赖）
: dashboard         独立看板 APK（依赖 :core 的 Provider 契约与数据模型）
```

模块	职责	依赖
`:core`	Room 实体/DAO/数据库、艾宾浩斯算法、词库解析、仓库、统计、Provider 契约	无 UI 依赖
`:app`	单 Activity + 9 个 Fragment、通知、WorkManager、ContentProvider 实现	`:core`
`:dashboard`	看板界面、自绘图表、CSV 导出、ContentResolver 读取	`:core`

4.2 分层架构

```
UI（Fragment / Adapter / ViewBinding）
        ↑ StateFlow<UiState>
ViewModel（协程 + viewModelScope）
        ↑ suspend / Flow
Repository（事务边界：导入、复习提交）
        ↑
Room DAO  ↔  SQLite
```

- **单向数据流**：Fragment 只调用 ViewModel 的方法、只渲染 `UiState`；ViewModel 不持有任何 View 引用；
- **Flow 驱动**：Room 的 `Flow` 查询让列表在数据变化时自动刷新，无需手动通知；
- **手写依赖容器**：项目只有 5 个仓库，用 `AppContainer` 显式构造比引入 Hilt 更直观，也便于讲清依赖关系。

4.3 关键设计决策

决策	理由
单 Activity + Navigation Component	页面切换统一由 `nav_graph.xml` 管理，返回栈行为一致；避免多 Activity 传参混乱
复习队列「进入时快照」	评分会修改 `dueAt`，若持续观察数据库，卡片会在答完后从列表中消失导致错位；快照保证一轮复习顺序稳定
词库导入放事务里	词库、单词、复习卡片必须同时写入，否则会出现「有词库没卡片 → 今日复习永远为空」的脏状态
看板用 ContentProvider 而非共享数据库	两个 APK 的应用沙箱互相隔离，不能直接读同一数据库文件；Provider + signature 权限是标准做法
图表自绘	课程要求「图表绘制」，用 Canvas 自绘既避免引入第三方库，也便于讲解绘制原理

5. 数据库设计

5.1 表结构

word_books（词库）

字段	类型	说明
id	INTEGER PK	自增
name	TEXT	词库名
description	TEXT	描述
source	TEXT	builtin / csv / txt / json
wordCount	INTEGER	词数
createdAt	INTEGER	创建时间（毫秒）

words（单词）

字段	类型	说明
id	INTEGER PK	自增
headword	TEXT	单词
phonetic	TEXT	音标
meaning	TEXT	中文释义
example / exampleCn	TEXT	例句及翻译
bookId	INTEGER	所属词库（索引）
familiar	INTEGER	是否熟悉
createdAt	INTEGER	创建时间

唯一索引 `(headword, bookId)`：同词库内不允许重复单词，重复导入会被 `OnConflictStrategy.IGNORE` 跳过。

review_cards（复习卡片）

字段	类型	说明
id	INTEGER PK	自增
wordId	INTEGER	外键 → words.id，级联删除，唯一索引
easeFactor	REAL	难度因子 EF（1.3 ~ 3.0）
intervalDays	INTEGER	当前间隔（天），0 表示学习阶段
repetitions	INTEGER	连续答对次数
lapses	INTEGER	遗忘次数
dueAt	INTEGER	到期时间（索引，用于今日队列查询）
lastReviewedAt	INTEGER	最近复习时间

state	INTEGER	0 新词 / 1 学习中 / 2 复习中 / 3 重学中

review_logs（复习日志）

字段	类型	说明
id	INTEGER PK	自增
wordId	INTEGER	索引
quality	INTEGER	1 / 3 / 4 / 5
prevIntervalDays / nextIntervalDays	INTEGER	间隔变化
reviewedAt	INTEGER	复习时间（索引，统计按天分组）

wrong_records（错题本）

字段	类型	说明
wordId	INTEGER PK	外键 → words.id，级联删除
wrongCount	INTEGER	累计错误次数
lastWrongAt	INTEGER	最近答错时间
mastered	INTEGER	是否已掌握

5.2 关系

```
word_books 1 ─── n words 1 ─── 1 review_cards
                    │ 1
                    ├── n review_logs
                    └── 1 wrong_records
```

5.3 关键查询

今日复习队列（联表一次取全，避免 N+1）：

```
SELECT w.*, c.id AS cardId, c.dueAt, c.repetitions, c.intervalDays,
       c.easeFactor, c.state, c.lapses
FROM review_cards c
INNER JOIN words w ON w.id = c.wordId
WHERE c.dueAt <= :now
ORDER BY c.dueAt ASC
```

按天统计（SQLite `strftime` 本地时区分组）：

```
SELECT strftime('%Y-%m-%d', reviewedAt/1000, 'unixepoch', 'localtime') AS day,
       COUNT(*) AS reviews,
       SUM(CASE WHEN quality >= 3 THEN 1 ELSE 0 END) AS correct
FROM review_logs
WHERE reviewedAt >= :from
GROUP BY day ORDER BY day ASC
```

6. 接口设计

6.1 内部接口：ContentProvider（主 App → 看板）

- authority：`com.wwdcqxt.wordmemory.provider`
- 权限：`com.wwdcqxt.wordmemory.permission.READ_STATS`，`protectionLevel="signature"`
- 只读：insert / update / delete 一律抛 `UnsupportedOperationException`

URI	返回	列
`content://.../overview`	单行	total_words、total_cards、learned_cards、mastered_cards、due_cards、total_reviews、correct_reviews、accuracy、today_reviews、today_accuracy、streak_days、wrong_count、updated_at
`content://.../daily`	多行	day、reviews、correct
`content://.../top_wrong`	多行	word_id、headword、meaning、wrong_count

权限选择说明：`signature` 级别的权限要求两个 APK 使用同一签名。

Android Studio 在同一台机器上的 debug 构建默认共用 `~/.android/debug.keystore`，所以直接按 debug 打包即可互通；

若换成正式签名，只要主 App 与看板用同一个 keystore 签名，权限依然自动授予。

6.2 外部接口：免费词典 API

- 地址：`https://api.dictionaryapi.dev/api/v2/entries/en/{word}`
- 无需 API Key，用于单词详情页「在线补充释义」；
- 请求超时 8 秒；
- 响应处理：200 解析音标与释义（取前 6 条）；404 → 提示未找到；其他状态码 / IO 异常 → 归类为「网络不可用」提示，页面保持可操作。

7. 关键实现

7.1 艾宾浩斯调度算法

`EbbinghausScheduler.next(state, quality, now)` 是纯函数，输出新的卡片状态与到期时间：

```
// ① 质量分 < 3：没记住，回到短期学习步
if (q < QUALITY_HARD) {
    val stepMinutes = if (q <= QUALITY_AGAIN) 10 else 30
    return ScheduleResult(..., intervalDays = 0, repetitions = 0,
```

```
        lapses = state.lapses + 1, state = STATE_RELEARNING,
        dueAt = now + stepMinutes * MINUTE_MS, ...)
}

// ② 成功：前两次固定 1 天、3 天，之后 interval * EF
val nextRepetitions = state.repetitions + 1
val interval = when {
    nextRepetitions == 1 -> 1
    nextRepetitions == 2 -> 3
    q == QUALITY_HARD -> maxOf(1, (state.intervalDays * ease * 0.6).roundToInt())
    else -> maxOf(1, (state.intervalDays * ease).roundToInt())
}.coerceAtMost(365)
```

难度因子按 SM-2 公式更新并限制在 `[1.3, 3.0]`：

```
val delta = 0.1 - (5 - q) * (0.08 + (5 - q) * 0.02)
return (current + delta).coerceIn(MIN_EASE, MAX_EASE)
```

为什么答错是 10 分钟而不是 1 天：背书场景下，遗忘的词需要在当天内重新见到，10 分钟后重新出现比隔天更符合记忆规律。

单元测试（`EbbinghausSchedulerTest`，8 个用例）覆盖：1 天/3 天固定步、EF 增长、答错重学、模糊间隔短于记得、EF 上下限、记忆强度单调性、非法质量分截断。

7.2 复习会话

- 进入页面时用 `first()` 取一次到期队列作为快照，避免评分过程中列表变化；
- 每次评分调用 `ReviewRepository.submitReview`，在一个事务里同时：更新卡片、写入日志、（质量 < 3 时）写入错题记录；
- `ReviewViewModel` 维护 `index / revealed / reviewedCount`，队列走完显示「本轮复习完成 + 剩余待复习数」。

7.3 词库导入

3. 通过 SAF（`ActivityResultContracts.OpenDocument`）选择文件，按扩展名判断格式；
4. `WordBookParser` 解析：CSV 支持引号转义与表头列名映射，TXT 支持制表符/连续空格，JSON 支持数组或 `{words:[...]}`；
5. 解析结果先展示「成功 N 个 / 跳过重复 M 个 / 前 5 条错误行」，用户确认后才写库；
6. 写库在 `withTransaction` 中完成：插入词库 → 插入单词（唯一索引去重）→ 为成功插入的单词创建复习卡片 → 回填词数。

7.4 异步与生命周期

- 所有仓库方法 `suspend`，DAO 查询返回 `Flow`；
- Fragment 用 `repeatOnLifecycle(STARTED)` 收集，页面停止时自动取消收集，避免泄漏；
- `debounce(250)` 处理搜索输入，避免每敲一个字母都查库；
- ViewModel 中 `Job` 可控（刷新时先取消旧任务再建新任务）。

7.5 权限与降级

场景	处理
Android 13+ 未授予通知权限	打开提醒开关时请求权限；拒绝则 Toast + 状态文案变为「已改为应用内提醒」，首页显示横幅，通知路径整体跳过
系统通知被手动关闭	`NotificationManagerCompat.areNotificationsEnabled() == false` 时同样降级
看板模块未安装	「打开数据看板」捕获 `ActivityNotFoundException`，提示未安装，不崩溃
网络不可用	在线释义提示「网络不可用」，本地释义与复习功能不受影响
文件无法读取	导入页 Toast 提示并保留原有状态

7.6 三态 UI

页面	加载态	空态	错误态
首页	居中 ProgressBar	文案 + 「加载内置词库」按钮	错误文案 + 重试
词库	居中 ProgressBar	空状态引导 + 内置词库按钮	文案
单词列表	RecyclerView 空列表	「没有匹配的单词」	同左
复习	进度条	队列为空直接显示「本轮复习完成」	异常文案
测试	生成题目中	「至少需要 4 个单词」	错误提示
看板	居中 ProgressBar	「还没有学习数据」	权限/异常分别提示 + 重试

7.7 每日提醒（WorkManager）

```
PeriodicWorkRequestBuilder<ReminderWorker>(1, TimeUnit.DAYS)
    .setInitialDelay(距离下一个 hh:mm 的毫秒数, TimeUnit.MILLISECONDS)
```

- 首次延迟精确到用户设定的时刻，之后每 24 小时一次；
- `ExistingPeriodicWorkPolicy.UPDATE`：用户改时间后立即生效；
- Worker 里先查到期数量，为 0 直接结束，避免打扰；通知不可用时静默返回。

8. 伴生模块设计（P2 数据看板）

8.1 独立性

- 独立 applicationId `com.wwdcqxt.wordmemory.dashboard`、独立图标、独立 Launcher；
- 不依赖主 App 的进程；主 App 未安装时打开看板会显示权限错误态而不是崩溃。

8.2 数据打通

```
主 App Room 数据库
      ↓（同进程查询）
StatsProvider（ContentProvider，signature 权限）
      ↓（ContentResolver.query）
StatsReader（看板侧）→ DashboardViewModel → UI
```

- 三个 URI 覆盖总览、每日趋势、错词榜；
- 所有查询在 IO 线程执行，异常统一转为错误游标，避免把看板拖崩。

8.3 图表绘制

组件	实现要点
`BarChartView`	三条网格线 + 圆角柱子 + 峰值数字 + 首/中/末日期标签；全 0 时画「暂无复习数据」
`DonutChartView`	背景环 + 从 12 点顺时针的前景弧 + 中心百分比与「正确率」说明

两个 View 都重写 `onDraw`，用 `Paint`/`Canvas`/`RectF` 绘制，不依赖任何图表库。

8.4 数据导出

- 生成带 UTF-8 BOM 的 CSV，Excel 打开中文不乱码；
- Android 10+ 用 MediaStore 写入「下载/WordMemory」，**无需申请存储权限**；
- Android 9 及以下写入应用专属外部目录，同样无需权限；
- 导出成功 Toast 显示具体路径。

8.5 产品化

项	做法
视觉	与主 App 区分色系（青绿），卡片化布局，统一圆角与字号
版本	`versionName 1.0.0`，`CHANGELOG.md` 记录 1.0.0 功能清单
文案	空态引导用户回主 App 操作，错误态区分权限与异常
稳定性	Provider 查询失败不崩溃；导出失败给出原因

9. 测试与问题记录

9.1 单元测试（21 个，全部通过）

测试类	数量	覆盖内容
`EbbinghausSchedulerTest`	8	固定步长、EF 增长、答错重学、模糊短间隔、EF 上下限、记忆强度、质量分截断

`WordBookParserTest`	8	CSV 表头映射、引号内逗号、两列 CSV、TXT 两种分隔符、JSON、去重、错误行、格式识别
`StudyStreakTest`	5	空数据、今天+昨天、今天缺勤、断档、0 次复习不计入

9.2 手工测试用例

编号	场景	预期
M1	首次安装启动	自动导入内置词库，首页出现到期单词
M2	复习一张卡片并选择「记得」	卡片从队列移除，详情页间隔变为 1 天
M3	复习选择「忘记」	10 分钟后重新到期，错题本 +1
M4	导入含错误行的 TXT	预览显示成功数与错误行，确认后只导入合法词条
M5	导入重复词库两次	第二次 skipped 计数增加，不产生重复单词
M6	拒绝通知权限后开启提醒	显示应用内横幅，无崩溃，WorkManager 仍可执行
M7	测试模式答错	错题本出现该词，错误次数 +1
M8	删除词库	词库、单词、卡片、日志一并删除，今日队列相应减少
M9	看板读取数据	卡片数字与主 App 一致，柱状图与正确率正确
M10	看板导出 CSV	下载目录出现文件，Excel 打开中文正常
M11	未安装主 App 打开看板	显示权限错误态并提供重试
M12	断网点击在线释义	提示网络不可用，详情页仍可正常浏览

9.3 开发中遇到的问题与解决

问题	原因	解决
`Granted` 构建首次失败：KSP 未生成 DAO 实现	使用 `api` 而非 `ksp` 声明 Room 编译器	改为 `ksp(libs.androidx.room.compiler)`
解析器编译报 `Unresolved reference 'addValidated'`	先写了调用、后写扩展函数，顺序错	补齐 `MutableList<ParsedWord>.addValidated` 扩展
单元测试断言失败（缺少释义提示里没有单词）	错误信息只带行号	错误文案补上原行内容，便于用户定位
导入 2 列 CSV 时释义被当成音标	无表头时按 3 列推断	按列数分支：≥3 列才有音标列
看板 `arrayOf(...)` 类型推断告警	Int/Float 混在一个数组	显式 `arrayOf<Any>(...)`

图表文字大小使用 `scaledDensity` 被标记过时	API 34 起废弃	改用 `TypedValue.applyDimension(COMPLEX_UNIT_SP, ...)`
`fallbackToDestructiveMigration()` 过时告警	Room 2.7 新增重载	改用 `fallbackToDestructiveMigration(dropAllTables = true)`

10. 总结与展望

完成情况：主 App 覆盖 B4 的全部核心功能与强制技术点，并额外做了网络请求与后台任务；伴生模块独立可运行、数据打通、图表自绘、导出可用；核心算法有单元测试保护。

可改进方向：

7. 目前是「到期即复习」，可以加入每日新词上限与混合队列（新词 + 复习）；
8. 看板可增加词库维度的对比与周报分享图；
9. 词库导入可支持 Excel（xlsx）与在线词库订阅；
10. 复习卡片可加入发音（TTS）与拼写测试题型。

附录

A. 目录结构

```
app/src/main/java/com/wwdcqxt/wordmemory/
├── WordMemoryApp.kt          Application：容器、通知渠道、首次导入
├── di/AppContainer.kt        手写依赖容器
├── data/                     DictionaryRepository、SeedWordBookImporter
├── notification/             NotificationHelper、ReminderScheduler、ReminderWorker
├── provider/StatsProvider.kt ContentProvider 实现
├── ui/                       MainActivity +
home/review/wordbook/wordlist/detail/quiz/wrong/settings/imports
└── util/TimeFormats.kt
core/src/main/java/com/wwdcqxt/wordmemory/core/
├── data/{entity,dao,db,model,repository}
├── domain/{algorithm,importer,stats}
└── provider/StatsContract.kt
dashboard/src/main/java/com/wwdcqxt/wordmemory/dashboard/
├── MainActivity.kt / DashboardViewModel.kt
├── chart/{BarChartView,DonutChartView}
├── data/{DashboardData,StatsReader}
└── export/CsvExporter.kt
```

B. 运行环境

- Android Studio（JBR 21）、Gradle 8.13、AGP 8.13.2、Kotlin 2.2.20
- compileSdk / targetSdk 36，minSdk 24
- 验证机型：Android 模拟器（API 36）与真机均可